Software Development Kit for Adding New Models to MODSIM

R. P. King

Department of Metallurgical Engineering

University of Utah

INTRODUCTION

MODSIM is a widely used simulation system for mineral processing plants. This simulator has an open-ended design which allows new models for ore dressing unit operations to be added whenever the need arises. This has proved to be one of the real strengths of MODSIM and many models have been added since it first became available. Most of the models that are now routinely used in the simulator were not in existence when it was first developed. Research into mineral processing operations will ensure a continuing supply of new modeling ideas well into the future and many of these will be incorporated into MODSIM either to provide enhanced simulation capabilities or simply to test the models over wide ranges of operating conditions and to test their ability to describe the operation of units in combination with other unit operations.

This software development kit provides two simple and intuitive interfaces which simplify the coding of the models and their introduction into MODSIM. It is based on Microsoft's Developer Studio which facilitates the creation of a dynamic link library that contains the code for the new models. It facilitates sharing unit models that are developed by others and accordingly provides considerable leverage for coding effort. Models can be distributed as dynamic linked libraries.

MODSIM uses a uniform interface for ore dressing unit models and provided that models adhere to this standard, they can be as simple or complex as required to provide a realistic description of the operating behavior of the unit that is being modeled. Models can range from simple regression models through to complex distributed state and distributed parameter models. Models can be easily modified and updated whenever required.

Note This manual includes instructions for writing models in Fortran and/or C/C++. This software development kit normally ships with only the Fortran code enabled so that users do not need to have both Fortran and C/C++ compilers available on their machines. If the C/C++ code is required please contact Minerals Technologies International Inc. for the alternative version. It is not possible to use the software development kit without a Fortran compiler installed.

ADDING A NEW MODEL TO MODSIM

The development of a new model requires two basic steps. Firstly the model must be defined and interpreted in computer code using Fortran or C/C++. This code is compiled into a dynamic link library. Secondly the new model must be notified to MODSIM and any unit parameters that will be set up using MODSIM's user interface during simulation must be described.

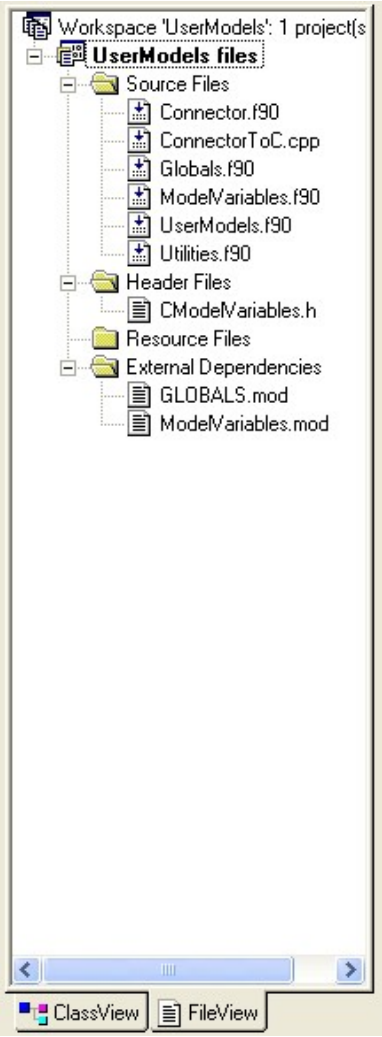

Figure 1 Workspace structure in Developer Studio that is used to build or modify the dynamic link library.

Using the Developer Studio to create the model code and build the dynamic link library.

A copy of Compaq Visual Fortran 6.5 must be installed on the development computer together with a compatible C/C++ compiler. Open Developer Studio and open the workspace UserModels.dsw from the Modsim installation directory. This workspace contains a number of files and modules which are shown in Figure 1. Each of the files in the workspace has a specific purpose and together they make up the UserModels dynamic link library.

```
Subroutine MODELSEQUENCE(Model)
!*******************************
  ! This SUBROUTINE is exported from UserModels.dll and is visible to MODSIM.DLL
  !MS$ ATTRIBUTES DLLEXPORT::MODELSEQUENCE

  USE GLOBALS

  Character*4 Model
  Character*5 cmodel
  Character*256 JobPath

  INTERFACE
    subroutine cmodelroutines(cmodel,JobPath)
      !DEC$ ATTRIBUTES C, ALIAS : '_cmodelroutines' :: cmodelroutines
      Character*5 cmodel
      !DEC$ ATTRIBUTES REFERENCE :: cmodel
      Character*256 JobPath
      !DEC$ ATTRIBUTES REFERENCE :: JobPath
    end subroutine
  END INTERFACE

  !Enter from MODSIM
  Select case (Model)
    Case ('BLBX')
    Call BLBX

    Case ('NAGE')
    Call NageswararaoCyclone

  !***** Add your Case and Call statements for Fortran model routines here.*****


    CASE DEFAULT
    cmodel = Model//char(0)
    Len = LEN_TRIM(UnitJobPath)
    JobPath(1:Len) = UnitJobPath(1:Len)
    JobPath(Len+1:Len+1) = char(0)

    Call cmodelroutines(cmodel,JobPath)

  End Select

  !Close the intermediate report files and the intermediate diagnostic file.
  Close(9)
  Close(14)

  !Return to MODSIM
End Subroutine MODELSEQUENCE
```

Figure 2 Code for file Connector.f90. Two connections to new model subroutines are shown.

Connector.f90 contains a single subroutine that is exported from the DLL and is visible to MODSIM. This subroutine provides the link between MODSIM and your user-written model subroutines. It is necessary to add two lines of code to this subroutine to identify your model Fortran subroutine to MODSIM. See the section Connect the Model Subroutine to MODSIM below for details.

C:\...\modsimdl\Globals.f90

```
  !This module is created by program DIMINP.FOR
  !It must not be changed by the user.
  MODULE GLOBALS
    !MS$ATTRIBUTES DLLEXPORT :: /VBVariables/
    Integer MaxOutputStreams,MaxInputStreams
    Real MeshSizes( 25)
    COMMON /Global/MaxOutputStreams,MaxInputStreams,MeshSizes

    CHARACTER*255 UnitJobPath
    INTEGER*4 UnitExitValue,UnitDiagFile
    COMMON /VBVariables/UnitExitValue,UnitDiagFile,UnitJobPath
  End MODULE GLOBALS
```

Figure 3 Code for module GLOBALS

Globals.f90 is a module that contains global variables that originate in MODSIM's graphics interface. These variables are

`MaxOutputStreams` = Maximum number of output streams that are allowed in a complete flowsheet (Integer)

`MaxInputStreams` = Maximum number of input streams that ar eallowed in a complete flowsheet (Integer)

`MeshSizes` = Boundaries of the size classes that MODSIM uses for internal calculations (Real vector)

`UnitExitValue` = Exit value (Integer)

C:\...\ModelVariables.f90

```
!This module is created by program DIMINP.FOR
!It must not be changed by the user.
MODULE MODELVARIABLES
  !MS$ ATTRIBUTES DLLEXPORT :: /MODELVARS/
  INTEGER UnitNumber
  LOGICAL Reporting
  Character*80 JobName
  CHARACTER*256 Message
  INTEGER NumSizeClasses,NumGClasses,NumSClasses
  REAL TotalSolidsF
  REAL TotalSolidsT, TotalSolidsC, TotalSolidsM
  REAL FeedWater
  REAL TailingsWater, ConcentrateWater, MiddlingsWater
  INTEGER NumberOfMessages,NumberOfMinerals,UnitType

  REAL GradeM( 22,  7), GradeV( 22,  7)
  REAL SolidSpGr( 22)
  REAL Texture( 50)
  REAL MagnSusceptG( 22)
  REAL OtherPropG( 22)
  REAL FlotnRateConst( 10)
  REAL MagnSusceptS( 10)
  REAL OtherPropS( 10)
  REAL CalValue( 22)
  REAL TotalSulfur( 22)
  REAL PyriticSulf( 22)
  REAL F_(  5502)
  REAL Feed( 25, 22, 10)
  EQUIVALENCE (F_(1),Feed(1,1,1))
  Real T_(  5502)
  REAL Tailing( 25, 22, 10)
  EQUIVALENCE (T_(1),Tailing(1,1,1))
  Real C_(  5502)
  REAL Concentrate( 25, 22, 10)
  EQUIVALENCE (C_(1),Concentrate(1,1,1))
  Real M_(  5502)
  REAL Middling( 25, 22, 10)
  EQUIVALENCE (M_(1),Middling(1,1,1))
  REAL Parameters(  50)
  REAL RepSize(  25)

  COMMON /MODELVARS/&
  UnitNumber,Reporting,NumSizeClasses,NumGClasses,NumSClasses,&
  UnitType,Feed,Tailing,Concentrate,Middling,&
  TotalSolidsF,TotalSolidsT,TotalSolidsC,TotalSolidsM,&
  FeedWater, TailingsWater, ConcentrateWater, MiddlingsWater,&
  Parameters,RepSize,NumberOfMessages,NumberOfMinerals,&
  GradeM,GradeV,&
  SolidSpGr,Texture,MagnSusceptG,OtherPropG,FlotnRateConst,&
  MagnSusceptS,OtherPropS,CalValue,TotalSulfur,PyriticSulf,&
  JobName
End MODULE MODELVARIABLES
```

Figure 4 Code for module ModelVariables. These variables enable information to be exchanged between Modsim and the user models.

`UnitDiagFile` = Logical unit number of the diagnostic file in the main DLL. (Integer)

`UnitJobPath` = Name of the path to the current directory (Character string)

These variables can be used in user models but, with the exception of `UnitExitValue` their values must not be altered in any model program. The file Globals.f90 itself must never be changed by the user.

ModelVariables.f90 contains the ModelVariables module that transmits necessary data from MODSIM to the user model and returns appropriate data from the user model to MODSIM. These variables are available for the user when constructing model subroutines and each variable is described below. File ModelVariables.f90 must never be changed by the user.

Variables that transmit data from MODSIM to the user models are:

`Feed(I,J,K)`	=	A 3-dimensional array that contains the mass flow rate of solids in size class I, grade class J and S-class K to the user model. This is the primary data required by any MODSIM models which all have the property of receiving a defined feed stream and returning the tailing, concentrate and middling stream as appropriate to the simulator which then passes this information on to the next connected model in the flow sheet. Flowrates are always in kg/s. (Real array)
`NumSizeClasses`	=	Number of size classes that are in use in the current simulation (Integer).
`NumGClasses`	=	Number of grade classes that are in use in the current simulation (integer).
`NumSClasses`	=	Number of S-classes that are in use in the current simulation (Integer).
`NumberOfMinerals`	=	Number of minerals in the system data.
`TotalSolidsF`		Total solid flowrate in the unit feed in kg/s (Real).
`FeedWater`		Total flowrate of water in the unit feed in kg/s (Real).
`RepSize`	=	Vector of representative sizes that are used internally by MODSIM in m (Real vector).
`Parameters`	=	Vector of parameters that define the operating condition for this unit. The parameters are defined in the software development kit and are specified by the user for each simulation in MODSIM's graphic user interface (Real vector).
`SolidSpGr`	=	Vector of specific gravities containing the specific gravities of the solid material in each grade class (Real vector).
`Texture`	=	Vector of parameters that characterize the mineralogical texture of the ore (Real vector).
`MagnSusceptG`	=	Vector of magnetic susceptibilities for the material in each G-class (Real vector).
`OtherPropG`	=	Vector of any arbitrary physical property associated with each G-class (Real vector).
`FltnRateConsts`	=	Vector of flotation rate constants. One for each s-class (Real vector).
`MagnSusceptS`	=	Vector of magnetic susceptibilities for material in each S-class (Real vector).
`OtherPropS`	=	Vector of any arbitrary physical property associated with each S-class (Real vector).

`CalVal`	=	Vector of calorific values for material in each G-class (Real vector) in J/kg.
`TotalSulfur`	=	Vector of total sulfur content of material in each G-class as mass percent (Real vector).
`PyriticSulf`	=	Vector of pyritic sulfur content of material in each G-class as mass % (Real vector).
`GradeM`	=	Matrix of distribution by mass of minerals in each grade class. `GradeM(J,M)` = mass fraction of mineral M in material in grade-class J (Real array).
`GradeV`	=	Matrix of distribution by volume of minerals in each grade class. `GradeV(J,M)` = volume fraction of mineral M in material in grade-class J (Real array).
`UnitNumber`	=	The number of the unit in the flowsheet.
`Reporting`	=	Logical variable that is set to `.TRUE.` when the model must send information to the report file.

The variables that are listed above represent a potentially rich collection of information about the material that is fed to a unit operation in the flowsheet. It is by no means necessary to use all of these variables in any particular model and only an appropriate subset will be used in any one case. Not all the physical properties will be defined in every simulation. For example it is unlikely that calorific values, and sulfur contents will be relevant in any but a coal washing plant.

The variables that are listed above should never be reset in a user model subroutine

Variables that are calculated by the model and returned to MODSIM

`Tailing(I,J,K)`	=	A 3-dimensional array that must be filled with the mass flowrate of solids in size class `I`, grade class `J` and S-class `K` in the tailing stream. Every model must produce at least a tailing stream since every unit operation in any flowsheet must have at least one product stream. Flowrates must be calculated in kg/s.
`Concentrate(I,J,K)`	=	A 3-dimensional array that can be filled with the mass flowrate of solids in size class `I`, grade class `J` and S-class `I`. This array is calculated only if the unit produces a concentrate. Flowrates are always in kg/s.
`Middling(I,J,K)`	=	A 3-dimensional array that must be filled with the mass-flowrate of solids in size class I, grade class J and S-class K. this array is calculated only if the unit produces a middling stream. Flowrates are always in kg/s.
`TotalSolidsT`	=	Total solid flowrate in the tailing stream, kg/s (Real).
`TailingsWater`	=	Total water flowrate in the tailing stream kg/s (Real).
`TotalSolidsC`	=	Total solid flowrate in the concentrate stream, kg/s (Real).
`ConcentrateWater`	=	Total water flowrate in the concentrate, kg/s (Real).
`TotalsolidsM`	=	Total solid flowrate in the middling stream, kg/s (Real).
`MiddlingWater`	=	Total water flowrate in the middling stream, kg/s (Real).

NumberOfMessages = Number of diagnostic messages sent for printing. This is set automatically and should not be set by the user.

The model subroutine must be constructed using these variables as the basic building blocks and placed in file UserModels.f90 or another convenient file that must be inserted in the project. At a minimum every model must calculate `Tailing(I,J,K)` for every relevant combination of `I`, `J`, and `K` as well as `TotalSolidsT` and `TailingsWater`.

```
//This file is created by program DIMINP.FOR
//It must not be changed by the user.

#pragma pack(2)
extern struct modsimvariables{
    int UnitNumber;
    int Reporting;
    int NumSizeClasses,NumGClasses,NumSClasses;
    int UnitType;
    float Feed[ 10][ 22][ 35];
    float Tailing[ 10][ 22][ 35];
    float Concentrate[ 10][ 22][ 35];
    float Middling[ 10][ 22][ 35];
    float TotalSolidsF;
    float TotalSolidsT, TotalSolidsC,TotalSolidsM;
    float FeedWater;
    float TailingsWater, ConcentrateWater, MiddlingsWater;
    float Parameters[ 50];
    float RepSize[ 35];
    int NumberOfMessages, NumberOfMinerals;
    float GradeM[  7][ 22], GradeV[  7][ 22];
    float SolidSpGr[ 22];
    float Texture[ 50];
    float MagnSusceptG[ 22];
    float OtherPropG[ 22];
    float FlotnRateConst[ 10];
    float MagnSusceptS[ 10];
    float OtherPropS[ 10];
    float CalValue[ 22];
    float TotalSulfur[ 22];
    float PyriticSulf[ 22];
    char JobName[80];
}MODELVARS;
#pragma pack()

#pragma pack(2)
extern struct VisualBasicVariables{
    long UnitExitValue;
    long UnitDiagFile;
    long UnitJobPath[255];
}VBVARIABLES;
#pragma pack()

char JobPathC[256];
```

Figure 5 Header file CmodelVariables.h that contains variable names for use in model functions that are written in C/C++

When programing in C remember that characters are case sensitive so variables must be spelled exactly as they appear in the header file CmodelVariables.h which is shown in Figure 5. The order of subscripting of arrays in C is the reverse of that in Fortran and array ranges are counted from 0 so

that, for example, the flowrate of solid material in the unit feed in size class i, grade class j and S-class k is referenced by `Feed[k-1][j-1][i-1]`

Every Fortran model subroutine should have the following two USE statements on the first two lines of the subroutine.

```
USE ModelVariables
USE GLOBALS
```

Utilities.f90 is a file containing a variety of utility routines. The user can add additional routines to this file. They will be visible to all subroutines in the UserModels.dll

```
#include <stdio.h>
#include <string.h>
#include "CModelVariables.h"

extern struct modsimvariables MODELVARS;
extern struct VisualBasicVariables VBVARIABLES;

void cmodelroutines( char *cmodel, char *JobPath);
void writeHeader(char *ModelType, char *model);

FILE *fptrRF;

void cbox();

void cmodelroutines( char *cmodel, char *JobPath)
{
    strcpy(JobPathC,JobPath);

    if(strcmp(cmodel,"CBOX")==0) cbox();

    //Insert your model subroutine call here

}
```

Figure 6 A single line of code handles the call to a model subroutine that is written in C/C++.

Connecting Fortran Model Subroutine to MODSIM

Add the following two lines of code to Connector

```
Case 'NNNN'
  Call SubrName
```

NNNN is the four-letter mnemonic that will identify the model in MODSIM. `SubrName` is the name chosen for the model subroutine.

Connecting C Model Subroutines to MODSIM

Models that are written in C must be compiled and linked into the UserModels.dll. A single line of code must be inserted into the file ConnectorToC.cpp as shown in Figure 6 . Add the following line

of code

```
if(strncmp(cmodel,"NNNN",4)==0) SubrName();
```

at the point indicated in Figure 6. NNNN is the 4-character mnemonic chosen to indentify the model in Modsim and SubrName is the name of the (void) procedure that defines the model.

Building the User Models Dynamic Link Library

Click Build → Build UserModels.dll in the Developer Studio. Fix any compile or link errors that are reported.

Writing the Report File for the New Model

The report file is a useful device to make it possible for the model subroutine to report the condition of the model when the flowsheet simulation calculation converges. The data that is reported depends on the model and this should be set up when the model subroutine is written. Code to write report information should be included in the model subroutine and executed when the variable `Reporting` is true. In Fortran subroutines report data is written to file with logical unit number `iounit` and in C programs to file pointed to by `fptrRF`.

Before writing any report information call the Reportheader Subroutine in Fortran and the writeHeader file in C. These routines have two arguments that define the type of unit and the 4-character mnemonic that identifies the model. The Fortran routine also returns the logical unit number of the report file.

NOTIFYING MODSIM OF THE NEW MODEL

Open the User Models Software Development Kit. Click the Model parameters tab as shown in

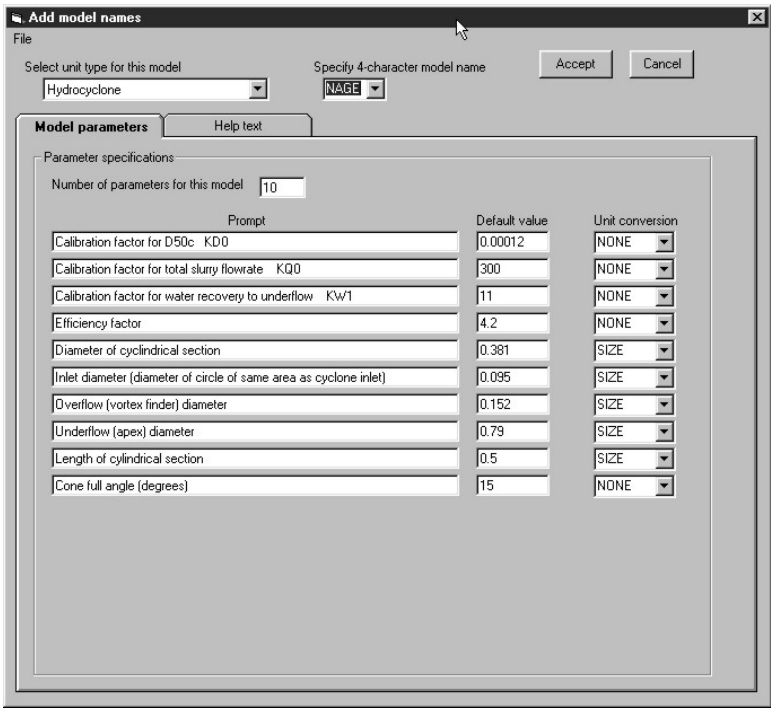

Figure 7 Specify the model mnemonic and the parameter prompts using the software development kit.

Figure 7. Select the unit type from the drop-down list to attach the model to. Specify the four-letter model mnemonic that identifies the model in MODSIM.

Specify the number of parameters that are used by the model and type the parameter prompts that you want to be displayed when editing model parameters in MODSIM. Specify a suitable default value for each parameter. Specify any unit conversion that should be displayed together with the parameter prompts. Possible selections are NONE, SIZE, FLOW, DENSITY.

Click the Help text tab and add the text of the help message for this model as shown in Figure 8. It is often helpful when using the model in MODSIM to have a list of the model parameters available in the help file. These can be added to the help editing window by clicking the Add parameters button.

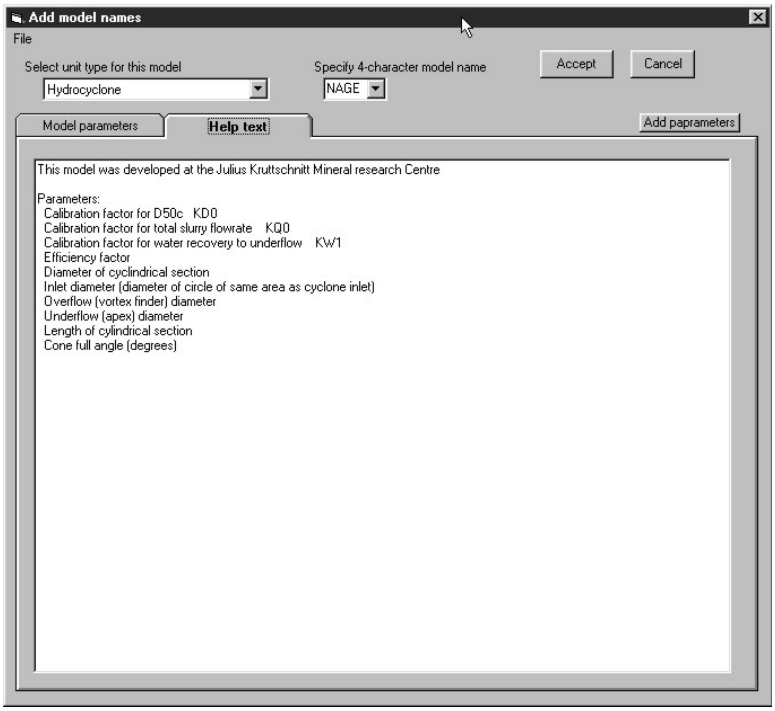

Figure 8 Window to add or edit the help message for the new model.

Accept and save these settings.

DEBUGGING MODEL SUBROUTINES

The recommended procedure for testing and debugging a new model is to set up a simple flowsheet in MODSIM that consists of only a single unit operation that can be described by the new model. Set up a suitable set of system data and run the flowsheet with one of the models for the unit type that is already included in Modsim. This will ensure that the system data that is being used is sensible. Note the performance and then change to the new model. The model should behave as expected in all respects. Change the model code to ensure that the code reflects accurately all the features of the original model formulation.

Errors can be tracked by writing error messages to the DLL diagnostic file from the model subroutine. The message must be written to character variable `Message` and following this with the call statement

```
Call Diagnostic(Message)
```

An example is shown in Figure 9 which shows a simple unit model subroutine in file UserModels.f90.

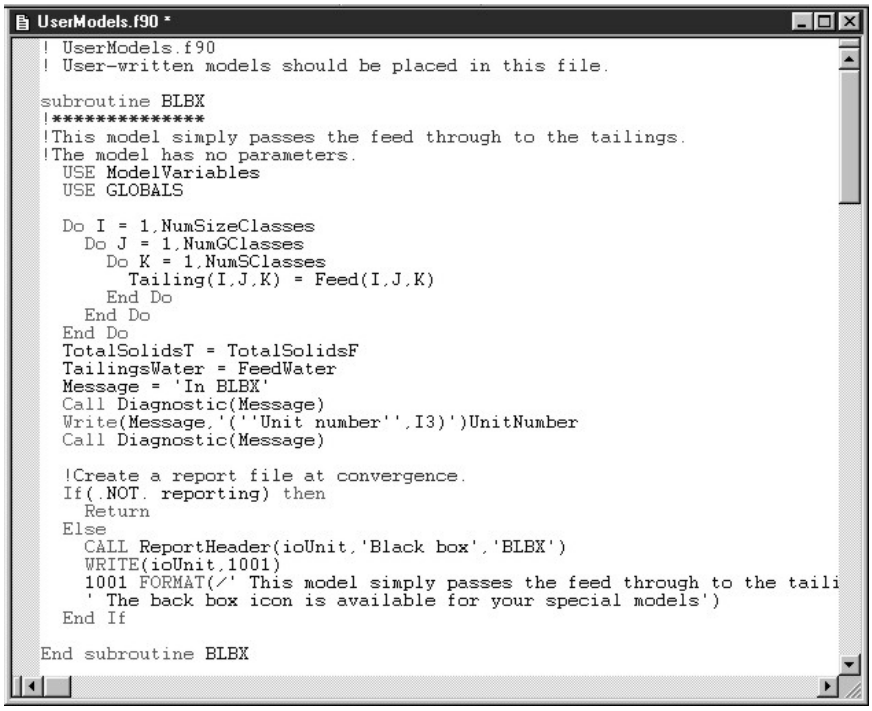

Figure 9 A simple model subroutine.

The diagnostic messages will show up in the DLL diagnostic file that is accessible from the Run menu on the MODSIM main form as shown in Figure 11. However if the code fails catastrophically in the model subroutine, the message will never reach the DLL diagnostic file. In this case the diagnostic messages will be available in the user model diagnostic file that is also accessible from MODSIM's Run menu as shown in Figure 11.

When programming in C/C++, diagnostic messages can be written using function `c_diagnostic` which takes the message text as a string for its argument. An example is shown in Figure 10.

```
void cbox()
{
    int i,j,k;
    float rec;

    strcpy(Cmessage, "In cbox");
    c_diagnostic(Cmessage);
    rec = MODELVARS.Parameters[0];
    MODELVARS.TotalSolidsT = 0;
    MODELVARS.TotalSolidsC = 0;

    for(i=0; i<MODELVARS.NumSizeClasses; i++)
    {
        for(j=0; j<MODELVARS.NumGClasses; j++)
        {
            for(k=0; k<MODELVARS.NumSClasses; k++)
            {
                MODELVARS.Tailing[k][j][i]=(float) rec*MODELVARS.Feed[k][j][i];
                MODELVARS.Concentrate[k][j][i]=(float) (1-rec)*MODELVARS.Feed[k][j][i];
                MODELVARS.TotalSolidsT += MODELVARS.Tailing[k][j][i];
                MODELVARS.TotalSolidsC += MODELVARS.Concentrate[k][j][i];
            }
        }
    }
    MODELVARS.TailingsWater = (float) rec*MODELVARS.FeedWater;
    MODELVARS.ConcentrateWater = (float) (1-rec)*MODELVARS.FeedWater;

    if(MODELVARS.Reporting)
    {
        writeHeader("Black box","CBOX");
        if(fptrRF)
        {
            fprintf(fptrRF,"This is a simple splitting model.\nIt is written in C and can be used as a template
            fprintf(fptrRF,"    Splitting factor to tailing = %7.3f\n",rec);
            fclose(fptrRF);
        }
    }
}
```

Figure 10 Example of a simple model written in the C language.

It is also possible to debug from within the Developer Studio if the compilation and build is done with debug information.

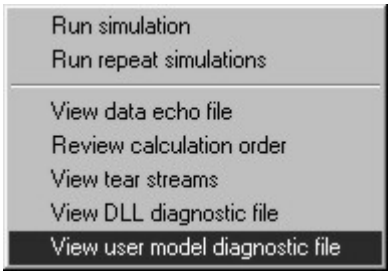

Figure 11 Modsim Run menu